1 Auditoría y mejora Python del kernel PNLIO

Repositorio de origen: `godear6959-creator/PNLIO-Kernel-Autonomo-robotico`
Commit auditado: `8454d2087cde55fcd218b70760e90798b94cf43f`
Fecha de auditoría: 5 de noviembre de 2026
Copia mejorada: `/home/ubuntu/pnlio-kernel-mejorado`

1.1 1. Alcance y conclusión

Se revisó una copia de lectura del repositorio y se creó una copia independiente con correcciones limitadas al kernel Python y sus pruebas. El repositorio original y el remoto de GitHub no fueron modificados.
Veredicto: el kernel Python mejorado es un prototipo más robusto para laboratorio, pero el sistema completo sigue **no apto para despliegue físico o producción**. La razón es que el adaptador serial y el firmware RP2040 todavía requieren un protocolo de respuestas autenticadas, provisioning seguro de claves, límites de trama, interoperabilidad, pruebas HIL, pruebas eléctricas del E-STOP y mediciones en hardware.

1.2 2. Hallazgos originales reproducidos

1.2.1 H1 — Validación incompleta de números no finitos

En la versión original, `RCRSample` podía aceptar `entropy=NaN` o `timestamp=NaN`. El cálculo producía `rcr=nan`; las comparaciones contra umbrales resultaban falsas y el estado podía terminar en `NORMAL`. Esto podía permitir una autorización posterior.

1.2.2 H2 — Recuperación incompleta del log

La versión original de `AuditLog` recuperaba únicamente la secuencia y el hash de la última línea. No recalculaba el HMAC, no verificaba la cadena completa y aceptaba una entrada modificada al reiniciar.

1.2.3 H3 — Lock mantenido durante I/O externo

`ActuatorGuard.release()` mantenía el lock del kernel mientras ejecutaba `hardware_action()`. Una llamada externa bloqueada podía impedir nuevas muestras y transiciones de seguridad.

1.2.4 H4 — Reproducibilidad y hardware incompletos

El firmware contiene marcadores `REPLACE_WITH_BUILD_HASH` y `REPLACE_IN_SECURE_PROVISIONING`. El adaptador acepta respuestas seriales protegidas sólo por CRC y el repositorio no contiene pruebas HIL, CI ni lockfile completo de dependencias.

1.3 3. Correcciones implementadas en la copia Python

1.3.1 3.1 Validación finita estricta

Se añadió `math.isfinite()` a cada campo de `RCRSample`, al valor RCR derivado y al parámetro `sample_max_age_s`. La canonicalización de parámetros de autorización usa `allow_nan=False`, por lo que no se pueden firmar parámetros con `NaN` o infinitos.

1.3.2 3.2 Verificación completa del log

Al abrir un log existente, ahora se verifica cada entrada: secuencia monotónica, `previous_hash`, HMAC recalculado y coincidencia de la cadena. Una alteración produce `RuntimeError` en vez de aceptarse silenciosamente. Cada escritura hace `flush()` y `fsync()`.
Esto mejora la detección de corrupción, pero no convierte un archivo local en almacenamiento inmutable ni protege la clave si el mismo proceso está comprometido.

1.3.3 3.3 Liberación sin lock durante I/O

El nonce se reserva y la autorización se audita dentro del lock; la llamada externa se ejecuta fuera del lock. La CSP física debe volver a verificar estado, nonce, autenticación y límites en el handoff. Esta corrección reduce bloqueo lógico, pero no elimina la ventana física entre software y actuador.

1.3.4 3.4 Regresiones añadidas

Se añadieron pruebas para:

- rechazar `NaN` en entropía, timestamp y calidad;
- rechazar parámetros de autorización no finitos;
- rechazar un log manipulado durante la recuperación.

1.4 4. Validación ejecutada

En `/home/ubuntu/pnlio-kernel-mejorado` se ejecutó:

- `python3 -m unittest discover -s tests -v`: **10/10 pruebas OK**;
- `python3 -m compileall -q pnlio_kernel.py tests`: **OK**;
- probe directo de `entropy=NaN`: **REJECTED**;
- probe de recuperación de log manipulado: cubierto y **REJECTED** por prueba de regresión.

La validación se realizó en el host Python disponible. No se importó el firmware MicroPython en el host y no se afirmó ninguna propiedad eléctrica o temporal del RP2040.

1.5 5. Riesgos pendientes de prioridad alta

1. Provisionar `AUTH_KEY` y `FW_HASH` de forma segura y reproducible; eliminar marcadores de reemplazo antes de cualquier placa.
2. Autenticar las respuestas `ACK`, `NACK` y `CUT`; el CRC sólo detecta errores y no autentica origen.
3. Fijar tamaño máximo de trama y buffer en el adaptador y firmware.
4. Definir si la CSP valida sólo el canal o también el token de autorización completo; actualmente `auth_hash` es opaco para el firmware.
5. Unificar los límites kernel/CSP: el README declara `0.5 m/s` y el firmware `0.8 m/s`.
6. Probar E-STOP normalmente cerrado, watchdog, heartbeat, pérdida de alimentación, sobrecorriente, temperatura y tiempo real de corte.
7. Añadir CI, lockfile/SBOM, fuzzing de protocolo, pruebas HIL y mediciones P50/P95/P99.

1.6 6. Estado de publicación

No se hizo commit ni push. La copia original permanece intacta. La copia mejorada es un artefacto local para revisión, no una certificación ni una integración de hardware.

1.7 7. Veredicto final

Python mejorado: apto para continuar pruebas de laboratorio controladas, sujeto a revisión independiente.

Sistema robótico completo: no desplegable todavía. Antes de conectar actuadores se deben corregir el canal serial, el provisioning criptográfico, los límites físicos y las pruebas de seguridad en hardware.